



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4802: Software Maintenance Lab

Summer, 2024-2025

Lab 2

Code Property Graph

Namisa Najah Raisa

210042112

Program: B.Sc in SWE

Department of CSE

Date: May 25, 2026

Contents

1	Lab 02 - An Intro to the Code Property Graph	2
1.1	What is CPG?	2
1.2	Installation	2
1.3	With joern-cli	4
1.4	With google colab	16
1.5	Ubuntu	19
1.6	My understanding	19

1 Lab 02 - An Intro to the Code Property Graph

1.1 What is CPG?

A Code Property Graph (CPG) is a way of representing source code as a graph - where every variable, function, statement, and condition becomes a node, and the relationships between them (like "this function calls that function" or "this variable flows into that one") become edges.

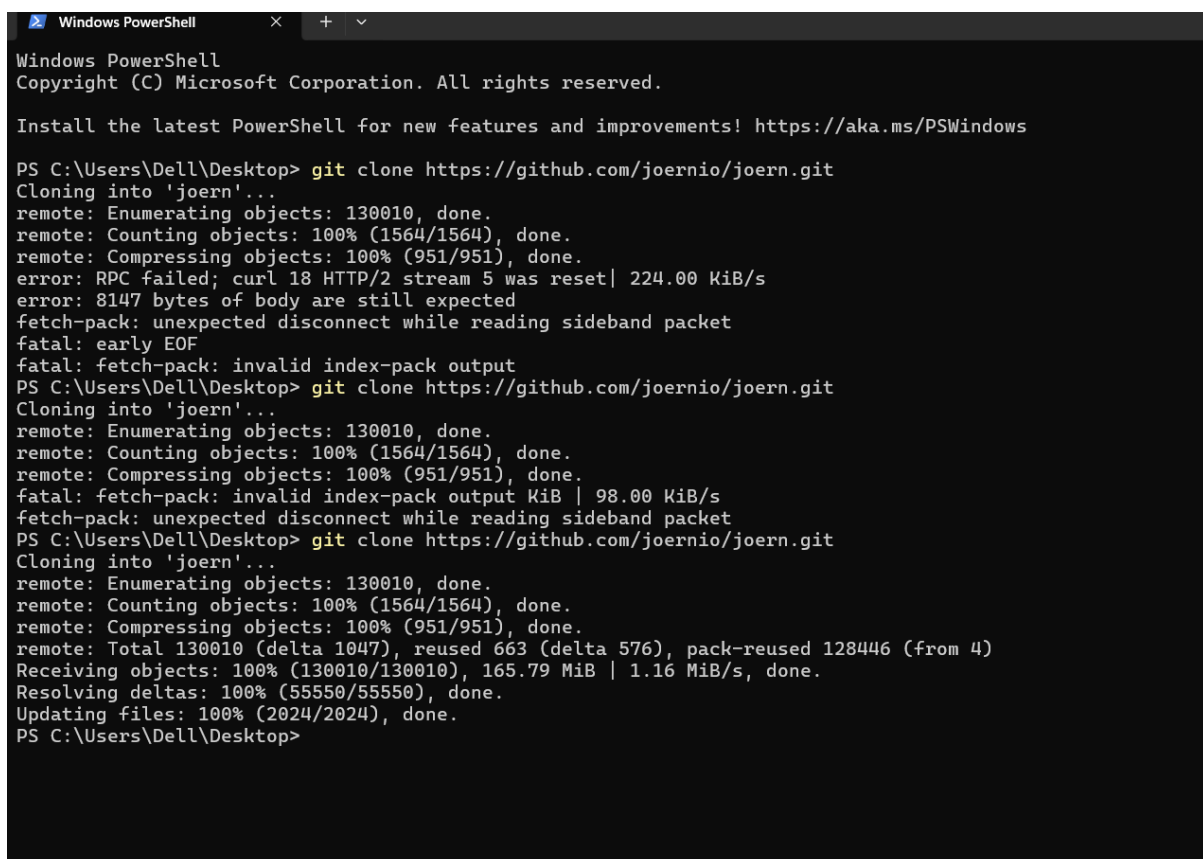
It combines three views of the code into one unified structure: the **AST (what the code looks like structurally)**, the **CFG (what order things execute in)**, and the **PDG (what depends on what)**. By merging all three, we can run graph queries to find patterns - like tracing where untrusted user input travels through the code until it hits a dangerous function, which is exactly how security vulnerabilities like buffer overflows or SQL injections get detected automatically.

1.2 Installation

1. Downloaded JDK version 26 first. It didn't work so I then changed it to version 21.
2. Clone the joern repository:

```
git clone https://github.com/joernio/joern.git
```

The cloning initially failed 3 times. It finally cloned on the 4th try.



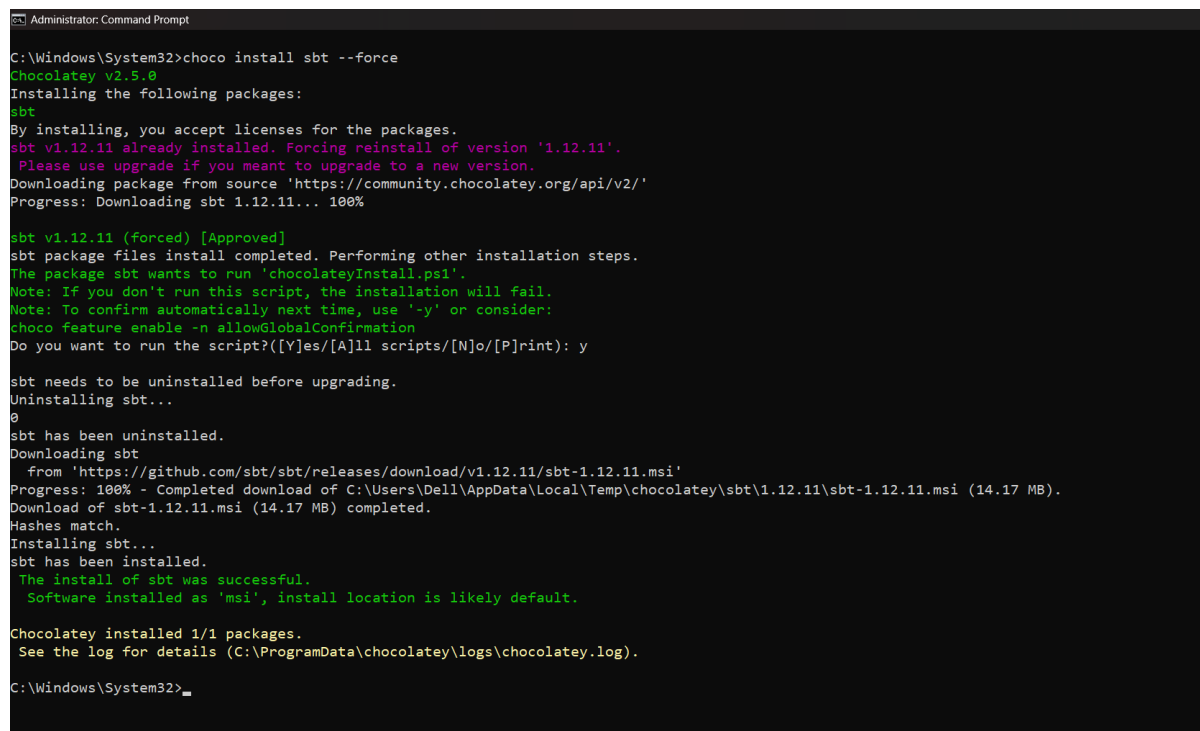
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Dell\Desktop> git clone https://github.com/joernio/joern.git
Cloning into 'joern'...
remote: Enumerating objects: 130010, done.
remote: Counting objects: 100% (1564/1564), done.
remote: Compressing objects: 100% (951/951), done.
error: RPC failed; curl 18 HTTP/2 stream 5 was reset | 224.00 KiB/s
error: 8147 bytes of body are still expected
fetch-pack: unexpected disconnect while reading sideband packet
fatal: early EOF
fatal: fetch-pack: invalid index-pack output
PS C:\Users\Dell\Desktop> git clone https://github.com/joernio/joern.git
Cloning into 'joern'...
remote: Enumerating objects: 130010, done.
remote: Counting objects: 100% (1564/1564), done.
remote: Compressing objects: 100% (951/951), done.
fatal: fetch-pack: invalid index-pack output KiB | 98.00 KiB/s
fetch-pack: unexpected disconnect while reading sideband packet
PS C:\Users\Dell\Desktop> git clone https://github.com/joernio/joern.git
Cloning into 'joern'...
remote: Enumerating objects: 130010, done.
remote: Counting objects: 100% (1564/1564), done.
remote: Compressing objects: 100% (951/951), done.
remote: Total 130010 (delta 1047), reused 663 (delta 576), pack-reused 128446 (from 4)
Receiving objects: 100% (130010/130010), 165.79 MiB | 1.16 MiB/s, done.
Resolving deltas: 100% (55550/55550), done.
Updating files: 100% (2024/2024), done.
PS C:\Users\Dell\Desktop>
```

Figure 1: git clone

```
choco install sbt
```



sbt stage

[illegible][illegible]

joern-install.sh —interactive

This instruction doesn't work with windows.

```

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dell\Desktop\joern>joern-install.sh --interactive

C:\Users\Dell\Desktop\joern>
[main 2026-05-25T06:20:01.191Z] StorageMainService: creating application shared storage
[main 2026-05-25T06:20:01.191Z] StorageMainService: creating application shared storage with local application storage fallback
[main 2026-05-25T06:20:01.197Z] CrossAppIPCServices: connecting to peer
[main 2026-05-25T06:20:01.420Z] [shared storage] Creating shared storage database at 'c:\Users\Dell\.vscode-shared\sharedStorage\state.vscdb' (wasCreated: false)
[main 2026-05-25T06:20:01.421Z] [shared storage] Initializing fallback application storage (type: local, path: c:\Users\Dell\AppData\Roaming\Code\User\globalStorage\state.vscdb)
[main 2026-05-25T06:20:01.535Z] [shared storage] Fallback application storage initialized with 275 items
[main 2026-05-25T06:20:01.543Z] update#setState idle
[main 2026-05-25T06:20:31.552Z] update#setState checking for updates
[main 2026-05-25T06:20:32.581Z] update#setState downloading
[main 2026-05-25T06:20:32.595Z] update#setState downloaded
[main 2026-05-25T06:20:32.595Z] update#setState ready
  
```

Figure 4: joern interactive shell install

Install neo4j desktop application.

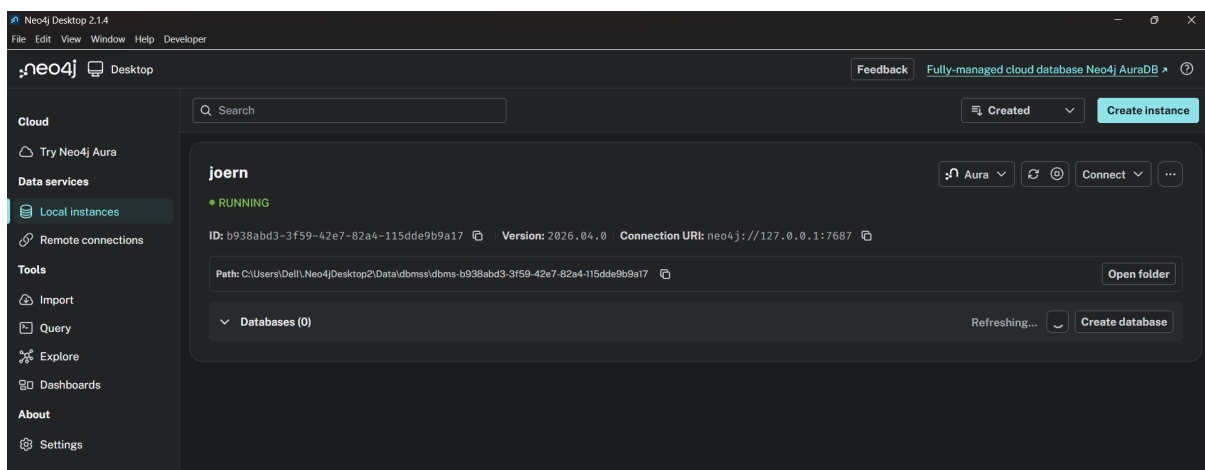


Figure 5

After the staging failed I looked for some other ways to do the task. Downloaded the Joern-cli.zip from the <https://github.com/joernio/joern/releases/tag/v4.0.547>.

1.3 With joern-cli

1. Create the code file. (test.cpp)

Listing 1: test code

```

1 void foo() {
2     int x=source();
3     if (x<MAX)
4     {
5         int y = 2 * x;
6         sink(y);
  
```

```
7     }  
8 }
```

```
C: > Users > Dell > Desktop > test.cpp  
  
1 void foo(){  
2     int x=source();  
3     if (x<MAX)  
4     {  
5         int y = 2 * x;  
6         sink(y);  
7     }  
8 }
```

Figure 6: The test code

2. Go inside the joern-cli folder and write the following commands:

```
joern-parse.bat C:\Users\Dell\Desktop\test.cpp  
—output cpg.bin //the code destination
```

3. The following command produced the csv files.

```
joern-export.bat cpg.bin —out cpg_export —repr  
all —format neo4jcsv
```

4. Need a code to process the csv files to import in neo4j to show the graph. Run this code (import_cpg.py):

Listing 2: import cpg code

```
1 from neo4j import GraphDatabase  
2 import os, csv  
3  
4 NEO4J_URI = "neo4j://127.0.0.1:7687"  
5 NEO4J_USER = "USER"  
6 NEO4J_PASSWORD = "PASSWORD"  
7 CPG_FOLDER = r"C:/Users/Dell/Desktop/joern-cli/joern-cli/  
8     cpg_export" //path to the joern-cli csv files  
9  
10 driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,  
11     NEO4J_PASSWORD))
```

```
11 def get_type(value):
12     try: return int(value)
13     except:
14         try: return float(value)
15         except: return value
16
17 def clean_row(row):
18     return {k: get_type(v) for k, v in row.items() if v != ""}
19
20 def read_csv(filepath):
21     with open(filepath, newline='', encoding='utf-8', errors='
22         ignore') as f:
23         return list(csv.DictReader(f))
24
25 def fix_headers(filepath):
26     with open(filepath, encoding='utf-8', errors='ignore') as f:
27         content = f.read()
28         if not content.strip():
29             return False
30         lines = content.splitlines()
31         headers = lines[0].split(',')
32         new_headers = []
33         seen = {}
34         for i, h in enumerate(headers):
35             h = h.strip().strip('"')
36             if not h:
37                 h = f'col_{i}'
38             if h in seen:
39                 seen[h] += 1
40                 h = f'{h}_{seen[h]}'
41             else:
42                 seen[h] = 0
43             new_headers.append(h)
44         lines[0] = ','.join(new_headers)
45         with open(filepath, 'w', encoding='utf-8') as f:
46             f.write('\n'.join(lines))
47         return True
48
49 def import_nodes(label, data_file):
50     if not fix_headers(data_file):
51         return 0
52     rows = read_csv(data_file)
53     if not rows: return 0
54     with driver.session() as session:
55         for row in rows:
56             props = clean_row(row)
57             if not props: continue
58             props_str = ", ".join([f"n. '{k}' = ${k}" for k in
59                 props])
60             if 'id' in props:
61                 query = f"MERGE (n: '{label}' {{{id': $id}}}) SET
```

```

        {props_str}"
    else:
        query = f"CREATE (n:'{label}') SET {props_str}"
        session.run(query, **props)
    return len(rows)

def import_edges(rel_type, data_file):
    if not fix_headers(data_file):
        return 0
    rows = read_csv(data_file)
    if not rows: return 0
    with driver.session() as session:
        for row in rows:
            props = clean_row(row)
            src = props.pop(':START_ID', None) or props.pop('start', None)
            dst = props.pop(':END_ID', None) or props.pop('end', None)
            if src is None or dst is None:
                keys = list(row.keys())
                src = get_type(row[keys[0]])
                dst = get_type(row[keys[1]])
            props_str = ""
            if props:
                props_str = "{" + ", ".join([f"'{k}': ${k}" for k in props]) + "}"
            query = f"""
                MATCH (a {{'id': $src}})
                MATCH (b {{'id': $dst}})
                MERGE (a)-[r:'{rel_type}' {props_str}]->(b)
            """
            session.run(query, src=src, dst=dst, **props)
    return len(rows)

# Clear existing data first
print("Clearing existing database...")
with driver.session() as session:
    session.run("MATCH (n) DETACH DELETE n")
print("Cleared\n")

all_files = sorted(os.listdir(CPG_FOLDER))
node_files = [f for f in all_files if f.startswith("nodes_") and f.endswith("_data.csv")]
edge_files = [f for f in all_files if f.startswith("edges_") and f.endswith("_data.csv")]

print(f"Found {len(node_files)} node types, {len(edge_files)} edge types\n")

print("=== Importing Nodes ===")
for filename in node_files:

```

```
104     label = filename.replace("nodes_", "").replace("_data.csv",
105         "")
106     print(f"    {label} ...", end=" ", flush=True)
107     try:
108         count = import_nodes(label, os.path.join(CPG_FOLDER,
109             filename))
110         print(f"        ({count})")
111     except Exception as e:
112         print(f"        {e}")
113
114 print("\n=== Importing Edges ===")
115 for filename in edge_files:
116     rel_type = filename.replace("edges_", "").replace("_data.csv",
117         "", "")
118     print(f"    {rel_type} ...", end=" ", flush=True)
119     try:
120         count = import_edges(rel_type, os.path.join(CPG_FOLDER,
121             filename))
122         print(f"        ({count})")
123     except Exception as e:
124         print(f"        {e}")
125
126 driver.close()
127 print("\n Done!")
```

```
CPG_FOLDER      = r"C:/Users/Dell/Desktop/joern-cli/joern-cli/cpg_export"

driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

✓ def get_type(value):
    try: return int(value)
✓     except:
        try: return float(value)
        except: return value

✓ def clean_row(row):
    return {k: get_type(v) for k, v in row.items() if v != ""}

✓ def read_csv(filepath):
✓     with open(filepath, newline='', encoding='utf-8', errors='ignore') as f:
        return list(csv.DictReader(f))

✓ def fix_headers(filepath):
✓     with open(filepath, encoding='utf-8', errors='ignore') as f:
        content = f.read()
✓     if not content.strip():
        return False
        lines = content.splitlines()
        headers = lines[0].split(',')
        new_headers = []
        seen = {}
✓     for i, h in enumerate(headers):
        h = h.strip().strip('""')
✓         if not h:
            h = f'col_{i}'
✓         if h in seen:
            seen[h] += 1
            h = f'{h}_{seen[h]}'
✓         else:
            seen[h] = 0
            new_headers.append(h)
        lines[0] = ','.join(new_headers)
```

Figure 7

```

PS C:\Program Files\Microsoft VS Code> & C:\Users\ DELL\AppData\Local\Programs\Python\Python311\python.exe c:/Users/ DELL/Desktop/import_cpg.py
Clearing existing database...
✓ Cleared

Found 18 node types, 16 edge types

=== Importing Nodes ===
BINDING ... ✓ (0)
BLOCK ... X {neo4j_code: Neo.ClientError.Statement.SyntaxError} {message: Invalid input '-': expected an identifier, an integer value or an octal integer value (line 1, column 86 (offset: 85))}
"CREATE (n:BLOCK) SET n.`25769803776` = $25769803776, n.`BLOCK` = $BLOCK, n.`-1` = $-1, n.`col_3` = $col_3, n.`col_4` = $col_4, n.`{` = ${"
^} {gql_status: 42001} {gql_status_description: error: syntax error or access rule violation
- invalid syntax
CALL ... X {neo4j_code: Neo.ClientError.Statement.SyntaxError} {message: Invalid input '-': expected an identifier, an integer value or an octal integer value (line 1, column 83 (offset: 82))}
"CREATE (n:CALL) SET n.`30064771072` = $30064771072, n.`CALL` = $CALL, n.`-1` = $-1, n.`x=source()` = $x=source(), n.`9` = $9, n.`STATIC_DISPATCH` = $STATIC_DISPATCH, n.`2` = $2, n.`<operator>.assignment` = $<operator>.assignment, n.`<operator>.assignment_1` = $<operator>.assignment_1, n.`2_1` = $2_1, n.`col_16` = $col_16, n.`void` = $void"
^} {gql_status: 42001} {gql_status_description: error: syntax error or access rule violation -
invalid syntax
CONTROL_STRUCTURE ... X {neo4j_code: Neo.ClientError.Statement.SyntaxError} {message: Invalid input '-': expected an identifier, an integer value or an octal integer value (line 1, column 122 (offset: 121))}
"CREATE (n:CONTROL_STRUCTURE) SET n.`47244640256` = $47244640256, n.`CONTROL_STRUCTURE` = $CONTROL_STRUCTURE, n.`-1` = $-1, n.`col_3` = $col_3, n.`col_4` = $col_4, n.`if (x<MAX)` = $if (x<MAX)"
^} {gql_status: 42001} {gql_status_description: error: s
yntax error or access rule violation - invalid syntax
FILE ... X {neo4j_code: Neo.ClientError.Statement.SyntaxError} {message: Invalid input '<': expected an identifier, an integer value or an octal integer value (line 1, column 88 (offset: 87))}
"CREATE (n:FILE) SET n.`60129542144` = $60129542144, n.`FILE` = $FILE, n.`<empty` = $<empty>, n.`<empty>_1` = $<empty>_1, n.`test.cpp` = $test.cpp, n.`0` = $0"
^} {gql_status: 42001} {gql_status_description: error: syntax error or access rule violati
on - invalid syntax
IDENTIFIER ... ✓ (5)
LITERAL ... ✓ (0)
LOCAL ... X {neo4j_code: Neo.ClientError.Statement.SyntaxError} {message: Invalid input 'x': expected an expression, ',', 'ORDER BY', 'CALL', 'CREATE', 'LOAD CSV', 'DELETE', 'DETACH', 'FILTER', 'FINISH', 'FOR', 'FOREACH', 'INSERT', 'LET', 'LIMIT', 'MATCH', 'MERGE', 'NEXT', 'NODETACH', 'OFFSET', 'OPTIONAL', 'REMOVE', 'RETURN', 'SET', 'SHOW', 'SKIP', 'TE

```

Figure 8: Terminal output

```

RMINATE', 'UNION', 'UNWIND', 'USE', 'WITH' or <EOF> (line 1, column 93 (offset: 92))
"CREATE (n:LOCAL) SET n.`94489280512` = $94489280512, n.`LOCAL` = $LOCAL, n.`int x` = $int x, n.`9` = $9, n.`<empty` = $<empty>, n.`2` = $2, n.`x` = $x, n.`1` = $1, n.`int` = $int"
^} {gql_status: 42001} {gql_status_description: error: syntax error or access rule vi
olation - invalid syntax
META_DATA ... ✓ (0)
METHOD_PARAMETER_IN ... ✓ (7)
METHOD_PARAMETER_OUT ... ✓ (7)
METHOD_REF ... ✓ (0)
METHOD_RETURN ... ✓ (7)
METHOD ... X {neo4j_code: Neo.ClientError.Statement.SyntaxError} {message: Invalid input '<': expected a node label/relationship type name, '$', '%' or '(' (line 1, column 115 (offset: 114))}
"CREATE (n:METHOD) SET n.`107374182400` = $107374182400, n.`METHOD` = $METHOD, n.`test.cpp:<global` = $test.cpp:<global>, n.`TYPE_DECL` = $TYPE_DECL, n.`void foo`() = $void foo()"
^} {gql_status: 42001} {gql_status_description: error: syntax e
rror or access rule violation - invalid syntax
NAMESPACE_BLOCK ... X {neo4j_code: Neo.ClientError.Statement.SyntaxError} {message: Invalid input '<': expected an identifier, an integer value or an octal integer value (line 1, column 123 (offset: 122))}
"CREATE (n:NAMESPACE_BLOCK) SET n.`137438953472` = $137438953472, n.`NAMESPACE_BLOCK` = $NAMESPACE_BLOCK, n.`<empty` = $<empty>, n.`<unknown` = $<unknown>, n.`<global` = $<global>, n.`<global>_1` = $<global>_1, n.`1` = $1"
^} {gql_status: 42001} {gql_status_description: error:
syntax error or access rule violation - invalid syntax
NAMESPACE ... ✓ (0)
TYPE_DECL ... X {neo4j_code: Neo.ClientError.Statement.SyntaxError} {message: Invalid input '<': expected a node label/relationship type name, '$', '%' or '(' (line 1, column 124 (offset: 123))}
"CREATE (n:TYPE_DECL) SET n.`167503724544` = $167503724544, n.`TYPE_DECL` = $TYPE_DECL, n.`test.cpp:<global` = $test.cpp:<global>, n.`TYPE_DECL_1` = $TYPE_DECL_1, n.`foo:void`() = $foo:void(), n.`1` = $1, n.`test.cpp` = $test.cpp, n.`foo:void`()_1` = $foo:void`()_1, n.`<empty` = $<empty>, n.`false` = $false, n.`1_1` = $1_1, n.`foo` = $foo, n.`1_2` = $1_2"
^} {gql_status: 42001} {gql_status_description: error:
syntax error or access rule violation - invalid syntax
TYPE ... ✓ (3)

```

Figure 9: Terminal output contd.

```

=== Importing Edges ===
ARGUMENT ... ✓ (8)
AST ... ✓ (65)
BINDS ... ✓ (0)
CALL ... ✓ (5)
CDG ... ✓ (6)
CFG ... ✓ (22)
CONDITION ... ✓ (0)
CONTAINS ... ✓ (31)
DOMINATE ... ✓ (21)
EVAL_TYPE ... ✓ (49)
PARAMETER_LINK ... ✓ (7)
POST_DOMINATE ... ✓ (21)
REACHING_DEF ... X {neo4j_code: Neo.ClientError.Statement.SyntaxError} {message: Unknown function 'source' (line 4, column 154 (offset: 232))}
"MERGE (a)-[r:REACHING_DEF]->(:30064771072:$30064771072,`124554051584`:$124554051584,`REACHING_DEF`:$REACHING_DEF,`x=source()`)->(:b)"
^} {gql_status: 42001} {
gql_status_description: error: syntax error or access rule violation - invalid syntax
REF ... ✓ (14)
SOURCE_FILE ... ✓ (10)
TRUE_BODY ... ✓ (0)

```

Figure 10: Terminal output contd.

Go to the neo4j instance and click import.

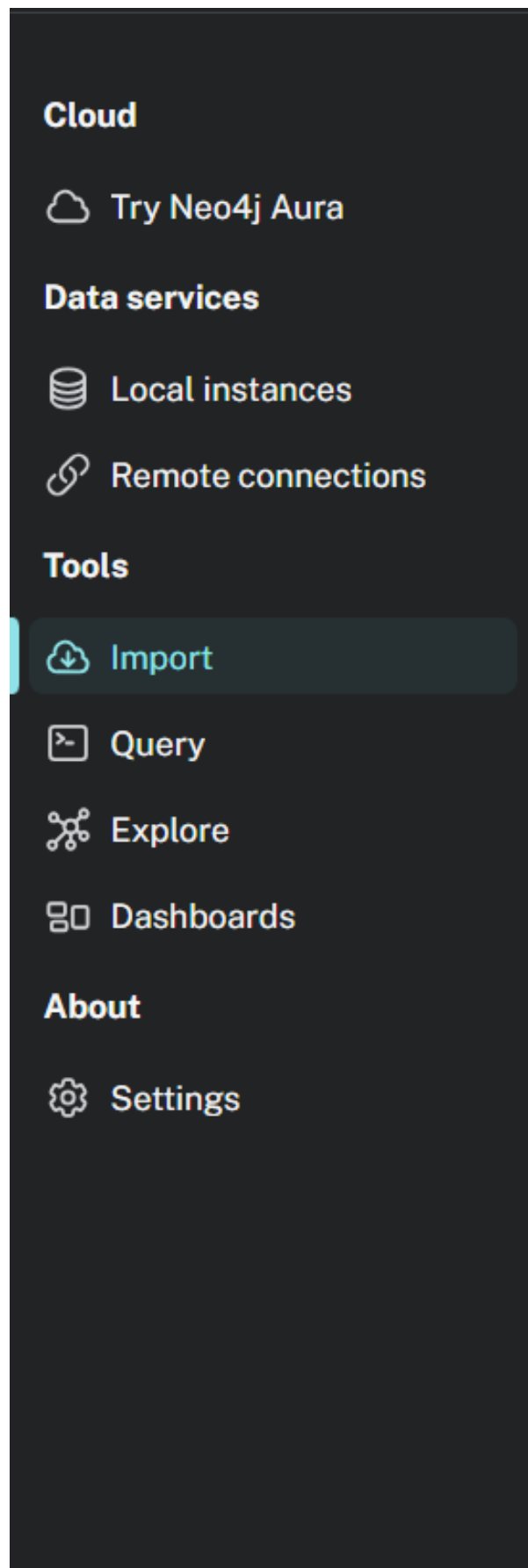


Figure 11

Had to upload the csv files created from the test code. And then it created the graph model of it.

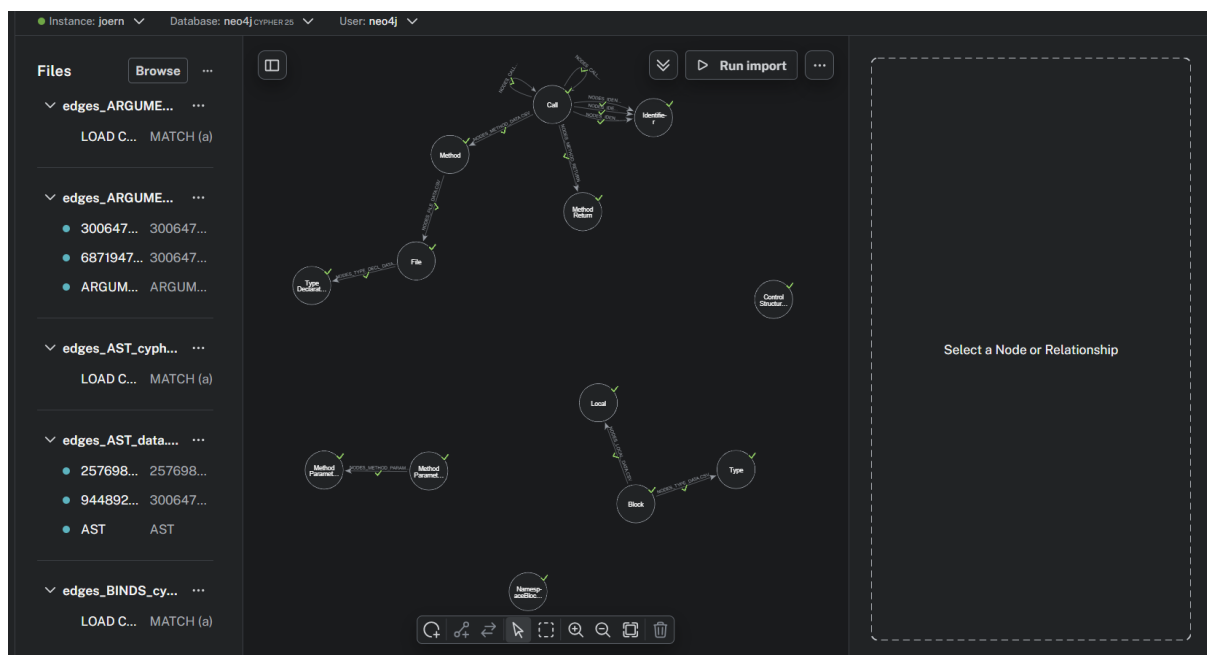


Figure 12: The graph model of the csv files.

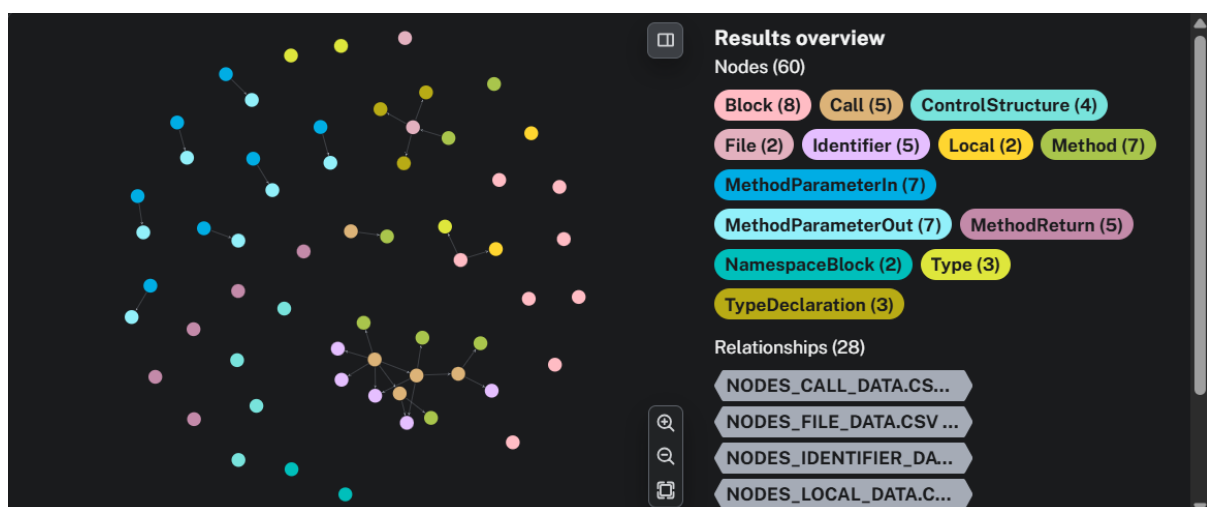


Figure 13



Figure 14



Figure 15

Explore tab under **import** shows the different types of layouts.

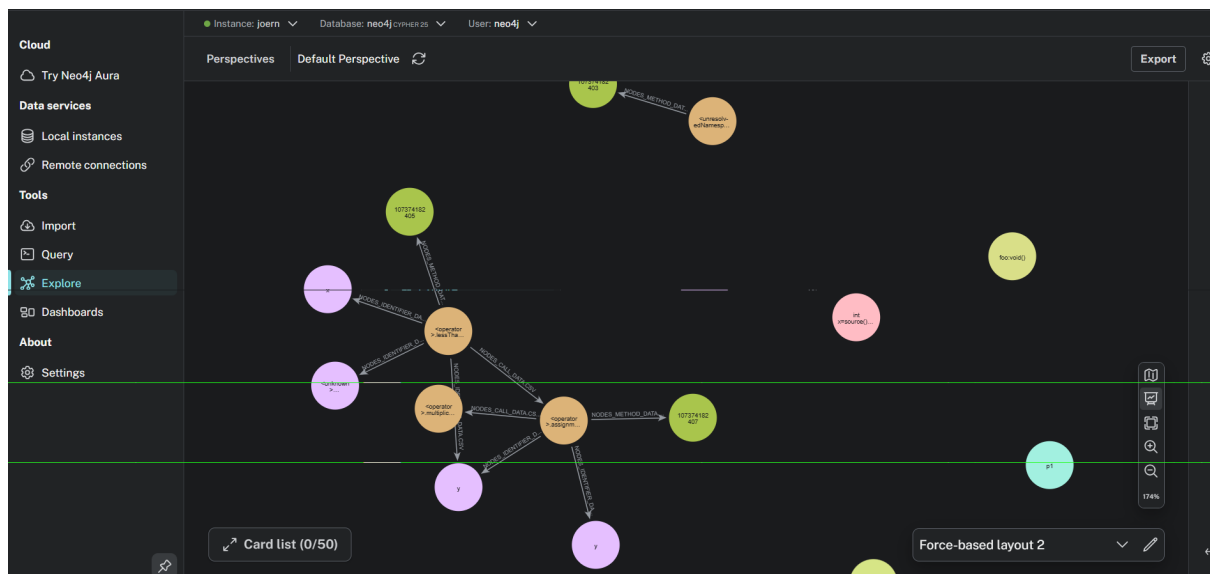


Figure 16: Forced based layout

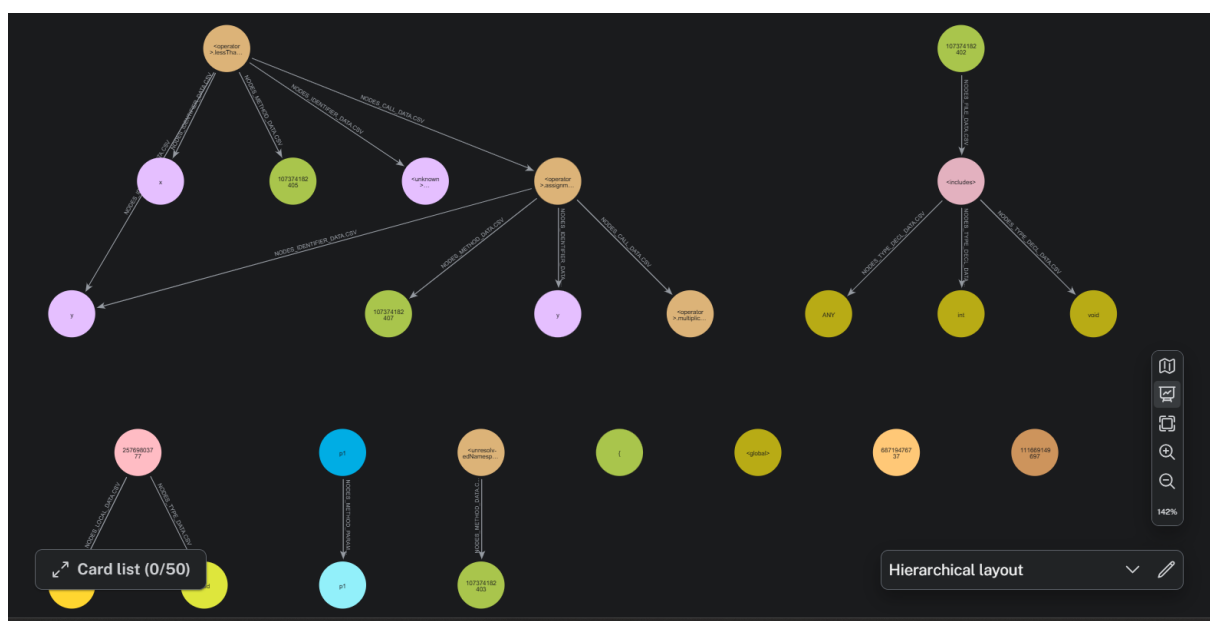


Figure 17: Hierarchical layout

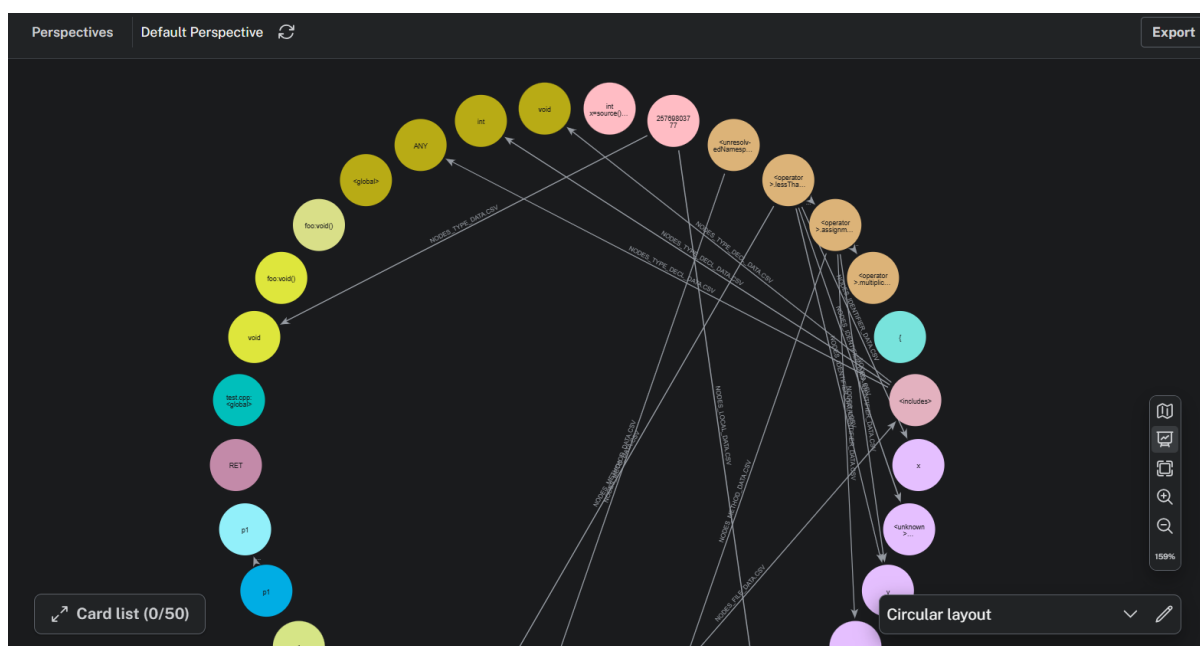


Figure 18: Circular layout

1.4 With google colab

Listing 3: cell 1

```

1 # Install Java
2 !apt-get update -qq
3 !apt-get install -y wget openjdk-21-jdk
4
5 # Set Java 21
6 !update-alternatives --set java /usr/lib/jvm/java-21-openjdk-amd64/
  bin/java
7
8 # Download installer
9 !wget -O joern-install.sh https://github.com/joernio/joern/releases/
  latest/download/joern-install.sh
10
11 # Make executable
12 !chmod +x joern-install.sh
13
14 # NON-INTERACTIVE install
15 !yes | ./joern-install.sh --install-dir /opt/joern
16
17 # Add to PATH
18 import os
19 os.environ["PATH"] += ":/opt/joern/joern-cli"
20
21 # Verify
22 !/opt/joern/joern-cli/joern-parse --version

```

Listing 4: cell 2

```
1
2 code_content = '''void foo(){
3     int x=source();
4     if (x<MAX)
5     {
6         int y = 2 * x;
7         sink(y);
8     }
9 }
10 '''
11
12 # Save to file
13 with open("source_code.cpp", "w") as f:
14     f.write(code_content)
15 print("    Source code saved to source_code.cpp")
```

Listing 5: cell 3

```
1 import subprocess
2 import os
3
4 # Parse the C++ source file
5 !joern-parse source_code.cpp -o cpg.bin
6 print("    CPG parsing complete")
```

Listing 6: cell 4

```
1 # Cell 4: Export CPG to Neo4j CSV format
2 !joern-export --repr=all --format=neo4jcsv /content/cpg.bin -o
   cpg_export
3 print("    CPG exported to Neo4j CSV format in 'cpg_export'
   directory")
```

Listing 7: cell 5

```
1 !zip -r cpg_export.zip cpg_export
```

Listing 8: cell 6

```
1 import os
2 import csv
3
4 folder = "/content/cpg_export"
5
6 for f in os.listdir(folder):
7     if f.endswith(".csv"):
8         path = os.path.join(folder, f)
9         print("\nFILE:", f)
10
11         try:
12             with open(path, newline='', encoding='utf-8', errors='
               ignore') as file:
```

```

13         reader = csv.reader(file)
14
15         for i, row in enumerate(reader):
16             print(row)
17             if i == 4:
18                 break
19
20     except Exception as e:
21         print("FAILED:", e)

```

Listing 9: cell 7

```

1 from google.colab import files
2 files.download("cpg_export.zip")

```

```

# Install Java
lapt-get update -qq
lapt-get install -y wget openjdk-21-jdk

# Set Java 21
update-alternatives --set java /usr/lib/jvm/java-21-openjdk-amd64/bin/java

# Download installer
wget -O joern-install.sh https://github.com/joernio/joern/releases/latest/download/joern-install.sh

# Make executable
chmod +x joern-install.sh

# NON-INTERACTIVE install
yes | ./joern-install.sh --install-dir /opt/joern

# Add to PATH
import os
os.environ["PATH"] += ":/opt/joern/joern-cli"

# Verify
!opt/joern/joern-cli/joern-parse --version

... W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide it (sources.list
Reading package lists... Done
Building dependency tree... Done

```

Figure 19: Google colab cell

```

[24] # Cell 3: Parse code to CPG
import subprocess
import os

# Parse the C++ source file
!joern-parse source_code.cpp -o cpg.bin
print("✓ CPG parsing complete")

... [0.001s][warning][os.container] Cgroup memory controller path at '/sys/fs/cgroup' seems to have moved to '/../jupyter-children', detected limits won't be accurate
[0.001s][warning][os.container] Cgroup cpu controller path at '/sys/fs/cgroup' seems to have moved to '/../jupyter-children', detected limits won't be accurate
Parsing code at: source_code.cpp - language: 'NEWC'
[+] Running language frontend

Invoking CPG generator in a separate process. Note that the new process will consume additional memory.
If you are importing a large codebase (and/or running into memory issues), please try the following:
1) exit joern
2) invoke the frontend: /opt/joern/joern-cli/c2cpg.sh -J-Xmx3244m source_code.cpp --output cpg.bin
3) start joern, import the cpg: 'importCpg("path/to/cpg")'

[+] Applying default overlays
Successfully wrote graph to: /content/cpg.bin
to load the graph, type 'joern /content/cpg.bin'
✓ CPG parsing complete

[25] # Cell 4: Export CPG to Neo4j CSV format

```

Figure 20: Google colab cell contd.

After running this code I got a `cpg_export` zip folder that contained all the csv files. And the next part is the same as the previous way (with `joern-cli`). I had to run the `import_cpg.py` code and then go to the `neo4j` import section to see the graph. The generated output was the same as the previous way.

1.5 Ubuntu

I also tried doing this task in **Ubuntu** since the interactive shell script should run on Linux, but it still didn't work.

1.6 My understanding

In this lab, I learned how a Code Property Graph (CPG) works by taking a simple C++ function - **foo()** - and turning it into a queryable graph database using Joern and Neo4j.

The code has a clear source → flow → sink pattern: data enters through `source()`, passes through an if condition, gets transformed into variable `y`, and finally reaches `sink(y)`. What the CPG graph makes visible is exactly how this data travels through the program - something we cannot easily see just by reading the code.